NiranjanaNS Portfolio

Version 1.0 | 2026-07-08 | Product Discovery and Design

1. Overview

NiranjanaNS Portfolio is an immersive software engineering portfolio that combines cinematic
storytelling with professional engineering proof. The product presents Niranjana’s journey,
skills, projects, architecture thinking, AI interests, and contact information through a polished,
responsive, accessible web experience.

2. Problem Statement

Recruiters and hiring managers review many portfolios quickly. Static portfolios often fail to
communicate engineering maturity because they focus on surface-level sections such as skills,
projects, and contact links.

This portfolio solves that problem by showing both:

- Immediate visual memorability.
- Deep technical credibility through case studies, architecture, and documentation.

3. Target Users

Recruiters

Recruiters need a fast understanding of who the candidate is, what role she fits, and whether her

work is worth forwarding to a technical team.

Hiring Managers

Hiring managers need proof of project ownership, problem solving, and communication skills.

Engineers and Interviewers

Engineers need evidence of architecture thinking, code quality, tradeoff awareness, and practical

technical depth.

Peers and Collaborators

Peers need a clear way to discover projects, GitHub repositories, demos, and areas of interest.

4. Product Goals

- Build a memorable portfolio experience without sacrificing clarity.
- Demonstrate frontend engineering through animation, interaction, responsiveness, and design.
- Demonstrate software engineering through architecture, case studies, documentation, and code.
- Make project exploration easy and useful.

- Present contact and resume actions clearly.
- Maintain high performance, accessibility, SEO, and best-practice scores.

5. Non-Goals

- Do not build an animation showcase that hides content.
- Do not depend on heavy assets that make the site slow.
- Do not use fake metrics, inflated skill percentages, or unclear claims.
- Do not make the mobile experience a reduced afterthought.
- Do not copy portfolio designs directly from inspiration sources.

6. Key User Journey

1. Visitor opens the portfolio.
2. Hero communicates name, role, and value proposition.
3. Visitor starts the journey.
4. Visitor travels through About, Education, Skills, Projects, Engineering, AI, and Contact.
5. Visitor explores one or more project case studies.
6. Visitor downloads resume or opens GitHub, LinkedIn, or email.

7. Functional Requirements

Hero

- Show name, role, tagline, and primary CTA.
- Include a minimalist developer character.
- Include subtle background motion.

- Provide quick access to resume, GitHub, LinkedIn, and email.

About

- Tell the personal story through concise timeline-style content.
- Show location, current focus, interests, and engineering values.

Education

- Present degree, institution, timeline, relevant learning areas, and current goals.

Skills

- Display skills as interactive crystals or similar visual objects.
- Avoid percentage bars.
- Each skill should expose related projects, libraries, experience notes, or favorite features.

Projects

- Serve as the largest and deepest section.
- Each project should include problem, solution, stack, architecture, features, screenshots,

GitHub link, live demo link, lessons learned, and future improvements.

- Highlight flagship projects first.

Engineering Hub

- Show architecture diagrams, folder structure, auth flow, API flow, testing approach,

performance strategy, caching strategy, deployment approach, and CI/CD plan where applicable.

AI Research Center

- Present AI-related work, experiments, integrations, and roadmap concepts.
- Clearly label planned ideas as roadmap, not completed work.

Achievements

- Present certifications, milestones, open-source contributions, hackathons, or measurable wins.

Contact

- Provide email, GitHub, LinkedIn, resume download, and a clear final CTA.

8. Non-Functional Requirements

Performance

- Target Lighthouse Performance score: 95+.
- Initial load should be under 2 seconds on a reasonable broadband connection.
- Heavy scenes and visual assets should be lazy-loaded.
- Respect reduced-motion preferences.

Accessibility

- Target Lighthouse Accessibility score: 95+.

- Use semantic HTML.
- Support keyboard navigation.
- Maintain visible focus states.
- Provide alt text for meaningful images.
- Avoid motion that blocks access to content.

SEO

- Target Lighthouse SEO score: 95+.
- Include meaningful metadata, Open Graph tags, sitemap, robots configuration, and structured

content where useful.

Responsiveness

- Support desktop, tablet, and mobile.
- Horizontal storytelling should adapt gracefully on small screens.
- Content must remain readable and navigable without relying on hover.

Maintainability

- Use a feature-based folder structure.
- Keep shared UI components separate from portfolio-specific sections.
- Store project and skill content in structured data files.
- Document major technical decisions.

9. Design Requirements

Visual Direction

Modern, premium, minimal, engineering-focused, and slightly cinematic.

Reference mood:

- Apple product storytelling.
- GitHub clarity.
- Linear polish.
- Vercel developer focus.
- A small cyber-inspired accent, without becoming cyberpunk.

Colors

- Background: #050816
- Surface: #0F172A
- Card: #111827
- Primary accent: #3B82F6
- Secondary accent: #7C3AED
- Success: #22C55E
- Warning: #F59E0B
- Error: #EF4444
- Text: #F8FAFC
- Muted text: #94A3B8

Typography

- Heading: Space Grotesk
- Body: Inter
- Code: JetBrains Mono

Spacing

Use an 8px spacing system:

8, 16, 24, 32, 40, 48, 64, 80, 96, 128

10. Technical Requirements

Planned Stack

- Next.js 15
- TypeScript
- Tailwind CSS
- Shadcn UI
- GSAP + ScrollTrigger
- Lenis
- Framer Motion
- Lucide React
- React Hook Form + Zod
- Vercel

Data Strategy

Use structured local data for:

- Navigation sections.
- Skills.
- Projects.
- Case studies.

- Achievements.
- Social links.

This keeps content maintainable and easier to update.

11. Risks

- Scroll animation can hurt usability if it becomes too complex.
- Heavy visual assets can reduce performance.
- Horizontal storytelling can be difficult on mobile.
- Too much motion can reduce accessibility.
- AI-generated UI may produce inconsistent design unless reviewed carefully.

12. Open Questions

- Which projects are confirmed as flagship case studies?
- Which resume file should be linked?
- Should the first release include sound, or should sound be a later enhancement?
- Should the first release include a full engineering notes page, or begin with notes inside each project?
- Should the project use a CMS later, or remain static for performance and simplicity?

13. Release Criteria

Version 1.0 is ready when:

- All core sections are implemented.
- At least three projects have complete case studies.

- Resume, GitHub, LinkedIn, and email actions work.
- Desktop and mobile layouts are polished.
- Lighthouse targets are met or documented with a remediation plan.
- README and docs explain the product clearly.